

CÓDIGOS

2) Ler um número inteiro e imprimir seu sucessor e seu antecessor.

```
#include <stdio.h>

int main(){
    int numero;

    printf("Digite um numero inteiro: ");
    scanf("%d", &numero);
    printf("Antecessor: %d\n", numero - 1);
    printf("Sucessor: %d\n", numero + 1);
    return 0;
}
```

3) Escreva um programa em linguagem C que calcule a autonomia de um veículo. O programa deve solicitar ao usuário a quantidade de combustível (em litros) no tanque e o consumo médio do veículo (em km/l). Com esses dados, o programa deve calcular e exibir quantos quilômetros o veículo ainda pode percorrer com o combustível disponível.

Exemplo de entrada:

Litros no tanque: 40

Consumo médio (km/l): 12

Saída esperada:

Autonomia: 480 km

```
#include <stdio.h>

int main(){
```

```

float litros, consumo, autonomia;

printf("Litros no tanque: ");

scanf("%f", &litros);

printf("Consumo medio (km/l): ");

scanf("%f", &consumo);

autonomia = litros * consumo;

printf("Autonomia: %.2f km\n", autonomia);

return 0;
}

```

4) Fazer um programa que solicite a largura e comprimento de terreno em metros e imprima a área do terreno. A área é o produto entre a largura e o comprimento,

```

#include <stdio.h>

int main(){

    float largura, comprimento, area;

    printf("Digite a largura do terreno: ");

    scanf("%f", &largura);

    printf("Digite o comprimento do terreno: ");

    scanf("%f", &comprimento);

    area = largura * comprimento;

    printf("Area do terreno: %.2f m2\n", area);

    return 0;

}

```

5) Faça um programa que solicite o número de elementos de vetor, solicite os elementos e armazene-os no vetor, e imprima a quantidade de elementos pares e ímpares

```
#include <stdio.h>
```

```
int main() {  
    int n, i, pares = 0, impares = 0;  
    printf("Quantidade de elementos: ");  
    scanf("%d", &n);  
    int vetor[n];  
    for(i = 0; i < n; i++) {  
        printf("Digite o elemento %d: ", i + 1);  
        scanf("%d", &vetor[i]);  
        if(vetor[i] % 2 == 0)  
            pares++;  
        else  
            impares++;  
    }  
    printf("Quantidade de pares: %d\n", pares);  
    printf("Quantidade de impares: %d\n", impares);  
    return 0;  
}
```

6) Desenvolver um algoritmo que leia dez números inteiro e verifique e imprima quantos são divisíveis por 5 e por 3 ao mesmo tempo.

```
#include <stdio.h>
```

```
int main() {  
    int numero, i, contador = 0;
```

```

for(i = 1; i <= 10; i++) {
    printf("Digite o %dº numero: ", i);
    scanf("%d", &numero);
    if(numero % 3 == 0 && numero % 5 == 0)
        contador++;
}
printf("Quantidade divisivel por 3 e 5 ao mesmo tempo: %d\n", contador);
return 0;
}

```

7) Fazer um programa que faz uma pesquisa com pessoas entre 18 e 80 anos. O programa deve solicitar a quantidade de pessoas a ser entrevistadas. Armazenar a idade dessas pessoas em um vetor e imprimir quantas pessoas de cada faixa etária foram entrevistadas de acordo com a tabela abaixo:

≥ 18 e < 35 = jovem

≥ 35 e < 65 = adulto

≥ 65 = idoso

O programa deve imprimir o quantitativo de jovens, adultos e idosos. Desta forma essas variáveis que irão contar deverão ser inicializadas com zero.

```
#include <stdio.h>
```

```

int main(){
    int qtd, i, idade;

    int jovens = 0, adultos = 0, idosos = 0;

    printf("Quantidade de pessoas entrevistadas: ");
    scanf("%d", &qtd);

    int idades[qtd];

    for(i = 0; i < qtd; i++) {

```

```

printf("Digite a idade da pessoa %d: ", i + 1);

scanf("%d", &idades[i]);

idade = idades[i];

if(idade >= 18 && idade < 35)

    jovens++;

else if(idade >= 35 && idade < 65)

    adultos++;

else if(idade >= 65)

    idosos++;

}

printf("Jovens: %d\n", jovens);

printf("Adultos: %d\n", adultos);

printf("Idosos: %d\n", idosos);

return 0;

}

```

8) Faça um programa que leia 10 números inteiros, armazene-os em um vetor, solicite um valor de referência inteiro e:

imprima os números do vetor que são maiores que o valor referência

retorne quantas vezes o valor de referência aparece no vetor

```
#include <stdio.h>
```

```

int main(){

    int vetor[10];

    int i, referencia, contador = 0;

    for(i = 0; i < 10; i++) {

        printf("Digite o numero %d: ", i + 1);
    }
}

```

```

        scanf("%d", &vetor[i]);
    }
    printf("Digite o valor de referencia: ");
    scanf("%d", &referencia);
    printf("Numeros maiores que o valor de referencia:\n");
    for(i = 0; i < 10; i++) {
        if(vetor[i] > referencia)
            printf("%d\n", vetor[i]);
        if(vetor[i] == referencia)
            contador++;
    }
    printf("O valor de referencia aparece %d vez(es).\n", contador);
return 0;
}

```

9) Códigos folhas dia 13/05

Código 1

```

#include <stdio.h>

// função para calcular média
float calcularmedia(float av1, float av2) {
    return (av1 + av2) / 2;
}

// função para verificar situação
void verificarsituacao(float media, char situacao[]) {
    if (media >= 7) {
        sprintf(situacao, "Aprovado");
    }
}

```

```

    }

    else if (media >= 5) {

        sprintf(situacao, "Recuperacao");

    }

    else {

        sprintf(situacao, "Reprovado");

    }

}

void cadastrarlunos(char nomes[][50], float av1[],

                    float av2[], float medias[],

                    char situacoes[][20], int qtd) {

int i;

for (i = 0; i < qtd; i++) {

    printf("\n--- Cadastro do Aluno %d ---\n", i + 1);


    printf("Nome: ");

    scanf(" %s", nomes[i]);


    printf("Nota AV1: ");

    scanf("%f", &av1[i]);


    printf("Nota AV2: ");

    scanf("%f", &av2[i]);

    // calcular média

    medias[i] = calcularmedia(av1[i], av2[i]);

    // verificar situação

    verificarsituacao(medias[i], situacoes[i]);

```

```
}  
}
```

```
void mostrartabela(char nomes[][50], float av1[],  
    float av2[], float medias[],  
    char situacoes[][20], int qtd) {  
    int i;  
  
    printf("\n===== TABELA DE ALUNOS =====\n");  
    printf("%-20s %-10s %-10s %-10s %-15s\n",  
        "NOME", "AV1", "AV2", "MEDIA", "SITUACAO");  
    printf("-----\n");  
    for (i = 0; i < qtd; i++) {  
        printf("%-20s %-10.2f %-10.2f %-10.2f %-15s\n",  
            nomes[i],  
            av1[i],  
            av2[i],  
            medias[i],  
            situacoes[i]);  
    }  
}
```

```
int main() {  
    int qtd;  
    printf("Digite a quantidade de alunos: ");  
    scanf("%d", &qtd);  
    char nomes[qtd][50];  
    float av1[qtd], av2[qtd], medias[qtd];
```



```

char situacoes[qtd][20];

// chamadas das funções

cadastrarlunos(nomes, av1, av2, medias, situacoes, qtd);

mostrartabela(nomes, av1, av2, medias, situacoes, qtd);

return 0;

}

```

Código 2

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define MAX 100


//=====

// FUNÇÃO PARA CALCULAR IMC

//=====

float calcularIMC(float peso, float altura) {

    return peso / (altura * altura);

}


//=====

// FUNÇÃO PARA CLASSIFICAR IMC

//=====

void classificarIMC(float imc, char classificacao[]) {

    if (imc < 18.5) {

        strcpy(classificacao, "Abaixo do peso");
    }
}

```

```

    }

    else if (imc < 25) {

        strcpy(classificacao, "Peso normal");

    }

    else if (imc < 30) {

        strcpy(classificacao, "Sobrepeso");

    }

    else if (imc < 35) {

        strcpy(classificacao, "Obesidade I");

    }

    else if (imc < 40) {

        strcpy(classificacao, "Obesidade II");

    }

    else {

        strcpy(classificacao, "Obesidade III");

    }

}

//=====

// FUNÇÃO PRINCIPAL

//=====

int main() {

    char nomes[MAX][100];

    int idades[MAX];

    float pesos[MAX];

    float alturas[MAX];

    float imcs[MAX];

    char classificacoes[MAX][50];

```

```

int i = 0;

char continuar = 'S';

printf("=====\n");

printf("      CALCULADORA DE IMC      \n");

printf("=====\n\n");

// CADASTRO COM WHILE
while ((continuar == 'S' || continuar == 's') && i < MAX) {

    printf("Paciente %d\n", i + 1);

    printf("Digite o nome: ");

    scanf(" %s", nomes[i]);

    printf("Digite a idade: ");

    scanf("%d", &idades[i]);

    printf("Digite o peso (kg): ");

    scanf("%f", &pesos[i]);

    printf("Digite a altura (m): ");

    scanf("%f", &alturas[i]);

    // VERIFICAÇÃO

    if (alturas[i] <= 0) {

        printf("Altura invalida!\n");

    }

    else {

        // CÁLCULO DO IMC

        imcs[i] = calcularIMC(pesos[i], alturas[i]);

        // CLASSIFICAÇÃO

        classificarIMC(imcs[i], classificacoes[i]);

```

```

        i++;
    }

    // CONTINUAR

    printf("\nDeseja cadastrar outro paciente? (S/N): ");

    scanf(" %c", &continuar);

    printf("\n");
}

// TABELA FINAL

printf("===== RELATORIO FINAL =====\n");
printf("%-20s | %-5s | %-8s | %-20s\n", "Nome", "Idade", "IMC", "Classificacao");
printf("-----\n");

int j = 0;
while (j < i) {

    printf("%-20s | %-5d | %-8.2f | %-20s\n", nomes[j], idades[j], imcs[j],
classificacoes[j]);

    j++;
}

printf("-----\n");

printf("\nPrograma encerrado!\n");

return 0;
}

```

Código 3

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define TAM 5 // -> DEFINICAO DE CONSTANTE (Anotação manuscrita)
```

```
//=====
```

```
// FUNÇÃO PARA CALCULAR DESCONTO
```

```
//=====
```

```
float calcularDesconto(float preco, float desconto) {
```

```
    float valorFinal;
```

```
    valorFinal = preco - (preco * desconto / 100);
```

```
    return valorFinal;
```

```
}
```

```
//=====
```

```
// FUNÇÃO PRINCIPAL
```

```
//=====
```

```
int main() {
```

```
    char produtos[TAM][50];
```

```
    float precos[TAM];
```

```
    float descontos[TAM];
```

```
    float valorFinal[TAM];
```

```
    int i = 0;
```

```
    char continuar = 'S';
```

```
    printf("=====\n");
```

```
    printf("        SISTEMA DE DESCONTO        \n");
```

```
    printf("=====\n\n");
```

```

while ((continuar == 'S' || continuar == 's') && i < TAM) {

    printf("Produto %d\n", i + 1);

    printf("Nome do produto: ");

    scanf(" %[^\\n]", produtos[i]);

    printf("Preco do produto: ");

    scanf("%f", &precos[i]);

    printf("Percentual de desconto: ");

    scanf("%f", &descontos[i]);


    valorFinal[i] = calcularDesconto(precos[i], descontos[i]);

    i++;


    printf("\nDeseja cadastrar outro produto? (S/N): ");

    scanf(" %c", &continuar);

    printf("\n");
}


printf("\n===== RELATORIO FINAL =====\n");
printf("-----\n");

printf("| %-15s | %-10s | %-10s | %-10s | \n", "Produto", "Preco", "Desconto", "Valor
Final");

printf("-----\n");

int j = 0;

while (j < i) {

    printf("| %-15s | %-10.2f | %-9.2f%% | %-10.2f | \n",

        produtos[j],

        precos[j],

        descontos[j],

```

```
        valorFinal[j]);  
        j++;  
    }  
    printf("-----\n");  
  
    return 0;  
}
```

10) Códigos dia 20/05

Código 1

```
#include <stdio.h>  
  
int main() {  
    int linhas, colunas;  
    int i, j;  
    int soma = 0;  
  
    // Leitura da quantidade de linhas e colunas  
    printf("Digite o numero de linhas da matriz: ");  
    scanf("%d", &linhas);  
    printf("Digite o numero de colunas da matriz: ");  
    scanf("%d", &colunas);  
  
    // Declaração da matriz após ler linhas e colunas  
    int matriz[linhas][colunas];
```

```
// Leitura dos elementos da matriz

printf("\nDigite os elementos da matriz:\n");

for(i = 0; i < linhas; i++) {

    for(j = 0; j < colunas; j++) {

        printf("Elemento [%d][%d]: ", i + 1, j + 1);

        scanf("%d", &matriz[i][j]);

        // Somatório dos elementos

        soma += matriz[i][j];

    }

}

// Impressão da matriz

printf("\nMatriz digitada:\n");

for(i = 0; i < linhas; i++) {

    for(j = 0; j < colunas; j++) {

        printf("%d\t", matriz[i][j]);

    }

    printf("\n");

}

// Impressão da soma

printf("\nSomatorio dos elementos = %d\n", soma);

return 0;

}
```


Código 2

```
#include <stdio.h>
```

```
// Função para ler a matriz
```

```
void lerMatriz(int linhas, int colunas, int matriz[linhas][colunas]) {
```

```
    int i, j;
```

```
    for(i = 0; i < linhas; i++) {
```

```
        for(j = 0; j < colunas; j++) {
```

```
            printf("Elemento [%d][%d]: ", i + 1, j + 1);
```

```
            scanf("%d", &matriz[i][j]);
```

```
        }
```

```
    }
```

```
}
```

```
// Função para imprimir a matriz
```

```
void imprimirMatriz(int linhas, int colunas, int matriz[linhas][colunas]) {
```

```
    int i, j;
```

```
    printf("\nMatriz:\n");
```

```
    for(i = 0; i < linhas; i++) {
```

```
        for(j = 0; j < colunas; j++) {
```

```
            printf("%d\t", matriz[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

// Função para calcular o somatorio

```
int somatorio(int linhas, int colunas, int matriz[linhas][colunas]) {  
    int i, j, soma = 0;  
    for(i = 0; i < linhas; i++) {  
        for(j = 0; j < colunas; j++) {  
            soma += matriz[i][j];  
        }  
    }  
    return soma;  
}
```

// Função para calcular o produtorio

```
int produtorio(int linhas, int colunas, int matriz[linhas][colunas]) {  
    int i, j, produto = 1;  
    for(i = 0; i < linhas; i++) {  
        for(j = 0; j < colunas; j++) {  
            produto *= matriz[i][j];  
        }  
    }  
    return produto;  
}
```

// Função para diagonal principal

```
void diagonalPrincipal(int linhas, int colunas, int matriz[linhas][colunas]) {  
    int i;  
    if(linhas != colunas) {  
        printf("\nA matriz nao e quadrada!\n");  
        return;  
    }  
}
```

```
}  
  
printf("\nDiagonal principal: ");  
for(i = 0; i < linhas; i++) {  
    printf("%d ", matriz[i][i]);  
}  
  
printf("\n");  
}
```

// Função para maior elemento

```
int maiorElemento(int linhas, int colunas, int matriz[linhas][colunas]) {  
    int i, j, maior = matriz[0][0];  
    for(i = 0; i < linhas; i++) {  
        for(j = 0; j < colunas; j++) {  
            if(matriz[i][j] > maior) {  
                maior = matriz[i][j];  
            }  
        }  
    }  
    return maior;  
}
```

// Função para menor elemento

```
int menorElemento(int linhas, int colunas, int matriz[linhas][colunas]) {  
    int i, j, menor = matriz[0][0];  
    for(i = 0; i < linhas; i++) {  
        for(j = 0; j < colunas; j++) {  
            if(matriz[i][j] < menor) {  
                menor = matriz[i][j];  
            }  
        }  
    }  
    return menor;  
}
```

```

    }
}
return menor;
}

```

// Função para buscar elemento

```

void buscarElemento(int linhas, int colunas, int matriz[linhas][colunas], int valor) {
    int i, j, encontrado = 0;
    for(i = 0; i < linhas; i++) {
        for(j = 0; j < colunas; j++) {
            if(matriz[i][j] == valor) {
                printf("\nElemento encontrado na posicao [%d][%d]\n", i, j);
                encontrado = 1;
            }
        }
    }
    if(!encontrado) {
        printf("\nElemento nao encontrado!\n");
    }
}

```

```

int main() {
    int linhas, colunas;
    int opcao;
    int valorBusca;

```

// Entrada das dimensões

```
printf("Digite o numero de linhas: ");
scanf("%d", &linhas);
printf("Digite o numero de colunas: ");
scanf("%d", &colunas);

int matriz[linhas][colunas];

// Leitura da matriz
lerMatriz(linhas, colunas, matriz);

do {
    printf("\n===== MENU =====\n");
    printf("1 - Imprimir matriz\n");
    printf("2 - Somatorio dos elementos\n");
    printf("3 - Produtorio dos elementos\n");
    printf("4 - Mostrar diagonal principal\n");
    printf("5 - Maior elemento\n");
    printf("6 - Menor elemento\n");
    printf("7 - Buscar elemento\n");
    printf("0 - Sair\n");
    printf("Escolha uma opcao: ");
    scanf("%d", &opcao);

    switch(opcao) {
        case 1:
            imprimirMatriz(linhas, colunas, matriz);
            break;
        case 2:
```

```
    printf("\nSomatorio = %d\n", somatorio(linhas, colunas, matriz));  
    break;  
case 3:  
    printf("\nProdutorio = %d\n", produtorio(linhas, colunas, matriz));  
    break;  
case 4:  
    diagonalPrincipal(linhas, colunas, matriz);  
    break;  
case 5:  
    printf("\nMaior elemento = %d\n", maiorElemento(linhas, colunas, matriz));  
    break;  
case 6:  
    printf("\nMenor elemento = %d\n", menorElemento(linhas, colunas, matriz));  
    break;  
case 7:  
    printf("\nDigite o elemento para busca: ");  
    scanf("%d", &valorBusca);  
    buscarElemento(linhas, colunas, matriz, valorBusca);  
    break;  
case 0:  
    printf("\nPrograma encerrado!\n");  
    break;  
default:  
    printf("\nOpcao invalida\n");  
}  
} while(opcao != 0);  
  
return 0;
```

